Chapter 5

Decision trees and random forests

In this chapter, we will be introducing a very powerful and popular method called *random forests*. In order to do so we first need to start with *decision trees*, since they are the fundamental building blocks of random forests: a random forest is 'set of trees', as we will see.

1. Decision trees

As usual, we start with the following setting:

$$\mathbf{x} = (x_1, ..., x_p) \rightarrow y$$

We need not place restrictions on the type of data we can handle with decision trees. That is to say, the input data can be either discrete or continuous. The same can be said for the output variable y. Hence, **decision trees (and random forests) can be used for both classification and regression tasks**. We first consider the problem of classification (i.e., with categorical output variable), before moving on to regression (i.e., quantitative output variable).

1.1. Classification task: Given a new (unseen) sample $\mathbf{x}^{\text{in}}$ with p descriptors, our objective is to predict its class $y \in \mathcal{C}_q = \{c_1, \ldots, c_Q\}$.

The intuition behind a decision tree is to partition the space of predictors and classify according to the splits - see Figure 5.1. Briefly, a decision tree will successively split the input space into regions. This can be visualised as a binary tree: each region is assigned a node, also known as *leaf nodes*. At any depth of the tree, the leaf nodes correspond to regions whose union gives us the full input space. The key aim is to **find a set of splits into regions separating samples of different classes into different regions**, while keeping samples of the same class in the same region (as much as possible).

Setting up the problem: Each split can be thought of as a *decision* that generates a tree. The split is strictly limited to <u>one</u> input variable at a time and can be defined by a

pair (j, s), where j is the input variable used for the decision and s is the threshold chosen:

$$\text{split } (j,s) = \left\{ \begin{array}{l} \{x_j : x_j < s\} =: R_1(j,s)\} \\ \{x_j : x_j \geq s\} =: R_2(j,s)\} \end{array} \right\}$$

which splits the space of input feature x_j into two regions $R_1(j, s)$ and $R_2(j, s)$.

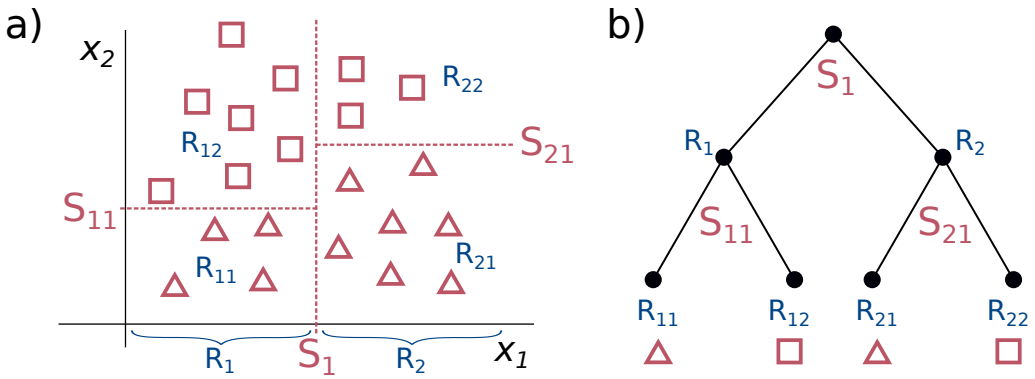


Figure 5.1. Visual intuition of a decision tree in a classification task. **a)** Using a simple case of two predictor variables as well as one categorical outcome with two classes, we can see that a decision tree is nothing other than a partition of the input variables' space. **b)** By computing different thresholds in a binary-tree-like manner, we find regions which contain only one class of data points. The corresponding prediction is denoted by the class symbols below.

Figure 5.1 shows us the intuition of how successive splits can be used to find regions for classification. In that case, we start with the S_1 split, which is only a 'decision' on the x_1 variable, denoted by $(j = 1, s = S_1)$. This is followed by S_{11} and S_{21}, which are solely based on x_2. If we choose the thresholds appropriately in each of the splits S_1, S_{11}, S_{21}, we find regions where we have a majority of triangles or squares. Indeed, it's almost always impossible to achieve a perfect partitioning , i.e., with 100% of cases in each leaf node being of a single class, and a majority rule will need to be applied.

Key message: A *decision tree* is given by the set of splits (j, s) associated with the branching points of the binary tree. Given a new sample, we use this binary tree to determine in which region the new instance falls, to give a prediction for the class based on the majority of samples in that region.

The learning task, in this case, is about making these successive decisions, i.e., choose a descriptor and find a threshold in that descriptor that will improve the tree's capability to separate the classes in the training set.

The algorithm. Since evaluating all possible combinations of splits across a number of levels of splits is a combinatorially and computationally intractable problem, **we utilise a *greedy* algorithm** that computes optimally one level at a time. In other words, the algorithm will first determine the first split optimally (i.e. S_1 in Figure 5.1), and then move on to the next level (i.e. S_{11} and S_{21}) to obtain optimal splits, but without reconsidering other possibilities for S_1. This helps keep the computational cost low and works quite well nonetheless in most cases. Note that in order to find an optimal split, the algorithm is not

limited to using any particular input variable, but rather chooses the optimal variable for the optimal split regardless of whether that variable was used before or not.

To determine the feature j and the threshold s for an optimal split at each step, we need to define a loss (or cost) function, just like we have been doing throughout the course.

The loss/cost function. The loss function is based on the classification quality achieved by a possible split (recall our discussion in Chapter 3 on the confusion matrix and the measure of its quality). Here, we will expand on those concepts with related measures imported from information theory.

We will need a few definitions. For a given region $R_{\boldsymbol{\alpha}}$ and a particular class c_q, the probability of being in that region and belonging to that class is:

$$\pi_q(R_{\boldsymbol{\alpha}}) = \frac{\sum_{i=1}^{N} I\big(\mathbf{x}^{(i)} \in R_{\boldsymbol{\alpha}} \wedge y^{(i)} \in c_q\big)}{\sum_{i=1}^{N} I(\mathbf{x}^{(i)} \in R_{\boldsymbol{\alpha}})}$$

This can now be computed for each class $q = 1, \ldots, Q$ and summarised in a $Q \times 1$ vector of probabilities:

$$\boldsymbol{\pi}(R_{\boldsymbol{\alpha}}) = \begin{bmatrix} \vdots \\ \pi_q(R_{\boldsymbol{\alpha}}) \\ \vdots \end{bmatrix}$$

where each component of the vector corresponds to the proportion of class q observations in the region (or node) with index α and is also called the *impurity* of node α.

Now, we can define the cost function for the splits in the decision tree. The two most broadly used measures of classification quality are the *Gini index* and the *cross-entropy*, which are both information theoretical measures. With these, **we want to achieve maximal *information* in our split**.

The Gini index is defined as follows, using notation from above:

$$\mathrm{GI}\big[\boldsymbol{\pi}(R_{\boldsymbol{\alpha}})\big] = \sum_{q=1}^{Q} \pi_q(R_{\boldsymbol{\alpha}})\big(1 - \pi_q(R_{\boldsymbol{\alpha}})\big)$$

Similarly, the cross entropy is defined as:

$$\mathrm{CE}\big[\boldsymbol{\pi}(R_{\boldsymbol{\alpha}})\big] = -\sum_{q=1}^{Q} \pi_q(R_{\boldsymbol{\alpha}}) \log \pi_q(R_{\boldsymbol{\alpha}})$$

which corresponds to the entropy of the distribution $\boldsymbol{\pi}(R_{\boldsymbol{\alpha}})$.

To better understand the logic of using these information-theoretic measures as optimisation criteria in machine learning, let us consider a simple case with $Q = 2$ classes and let us focus on only one region α (hence we will drop the index α below). We will call π the proportion of data in class '1', so that the proportion of data in class '2' is given by $(1 - \pi)$. In this simple case we then have:

$$\begin{aligned} \mathrm{GI}\big[\pi\big] &= 2\pi\big(1 - \pi\big) \\ \mathrm{CE}\big[\pi\big] &= -\pi \log \pi - \big(1 - \pi\big) \log \big(1 - \pi\big) \end{aligned}$$

Both these functions are plotted in Figure 5.2, showing that they are maximal when $\pi = 0.5$, i.e., when the proportion of data in one region is the same for all classes (see *Quick aside 1* for a brief illustration of this fact for the GI). Conversely, both the Gini index and CE decrease when π and $(1 - \pi)$ are more dissimilar, hence where the distribution of data points upon splitting increases the information about classes.

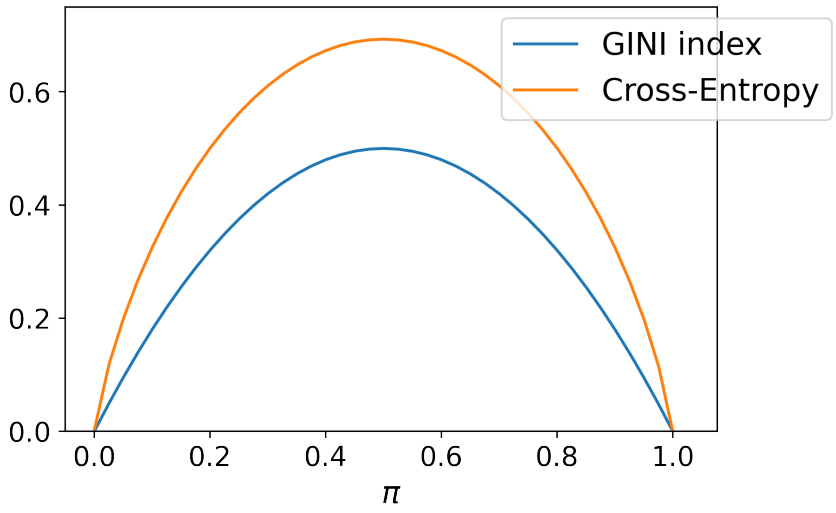


Figure 5.2. Gini index and entropy (or cross-entropy) for a two-class case.

Quick aside 1. Let us quickly illustrate how to show that the GI is maximised when all the p_i's are equal, i.e., for a uniform distribution across classes. For J classes, i.e. $i \in \{1, \dots, J\}$ and a corresponding probability vector $\mathbf{p} = (p_1, \dots, p_J)^T$, we have:

$$\mathrm{GI}(\mathbf{p}) = \sum_{i=1}^{J} p_i(1 - p_i) = 1 - \sum_{i=1}^{J} p_i^2$$

To show this, we use the method of Lagrange multipliers, a classic method to carry out optimisation of functions under constraints.

We will maximise GI under a constraint that enforces the normalisation of probabilities. To do so, we define a Lagrangian:

$$L = \mathrm{GI}(\mathbf{p}) - \lambda \left(\sum_{i=1}^{J} p_i - 1 \right)$$

and obtain the stationary point:

$$\left\{ \begin{array}{l} \nabla_{\mathbf{p}} L = -2\,\mathbf{p}^\star - \lambda^\star\,\mathbf{1} = \mathbf{0} \\ \dfrac{\partial L}{\partial \lambda} = \displaystyle\sum_{i=1}^{J} p_i^\star - 1 = \mathbf{1}^T \mathbf{p}^\star - 1 = 0 \end{array} \right\}$$

which gives finally

$$\mathbf{p}^\star = \frac{-\lambda^\star}{2}\mathbf{1} \quad \text{and} \quad \lambda^\star = \frac{-2}{J}$$
$$\implies p_i^\star = \frac{1}{J},\ \forall i$$

and one can further prove that this stationary point is a maximum.

Therefore, **the Gini index and the CE can be used to measure the deviation from a uniform probability distribution**, exactly like entropy, whose interpretation in terms of information is well established in information theory (and the CE is an entropy at the end of the day, see *Quick aside 2*). As such, these measures play the role of **functions to optimise in order to perform a classification task**, whose objective is to obtain a vector of class-assignment probability that is as concentrated on one class as possible.

(Note: it would be perfectly legitimate to use for this purpose classification metrics such as the misclassification error (given by 1-accuracy). In practice, however, the misclassification error has a few shortcomings compared to both the Gini index and cross-entropy (e.g., it is not differentiable everywhere) and hence is much less used, see e.g. Section 9.2 of Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*).

Quick aside 2. The terminology "cross-entropy" requires some clarification. Consider a certain region $R_{\boldsymbol{\alpha}}$, where the node impurity $\pi_q(R_{\boldsymbol{\alpha}})$ is given by (1.1). Let us denote by $\hat{\pi}_q(R_{\boldsymbol{\alpha}})$ the model probability that points in $R_{\boldsymbol{\alpha}}$ belong to class q. Then the cross-entropy between the *data distribution* and the *model distribution* is given by:

$$\mathrm{CE}\big[\boldsymbol{\pi}, \hat{\boldsymbol{\pi}}\big] = -\sum_{q=1}^{Q} \pi_q(R_{\boldsymbol{\alpha}}) \log \hat{\pi}_q(R_{\boldsymbol{\alpha}}) \tag{5.1}$$

Typically in machine learning the cross-entropy is used as a loss function to be minimised in order to set the optimal model distribution, because it is easy to show (try it as an exercise) that the CE is *minimal* when the model distribution is equal to the data distribution.
By substituting the model distribution $\hat{\boldsymbol{\pi}}$ with the data distribution $\boldsymbol{\pi}$ we then obtain:

$$\mathrm{CE}\big[\boldsymbol{\pi}(R_{\boldsymbol{\alpha}})\big] = -\sum_{q=1}^{Q} \pi_q(R_{\boldsymbol{\alpha}}) \log \pi_q(R_{\boldsymbol{\alpha}}) \tag{5.2}$$

This is the reason why the CE-based splitting criterion in decision trees consists eventually of minimising the entropy of the data distribution $\boldsymbol{\pi}(R_{\boldsymbol{\alpha}})$.

Implementation in the algorithm. *Quick Aside 1* above tells us that the Gini index (or the cross-entropy) is maximal when all probabilities in the vector are equal, i.e. no information was gained, which is precisely what we do not want. Therefore, **the greedy estimation algorithm will try to minimise the Gini index (or cross-entropy)**. This means that at given split that would produce two daughter nodes corresponding to the regions R_1 and R_2, we find (j, s) such that:

$$\min_{j,s} \Big[\mathbb{p}_{R_1} \mathrm{GI}\big(\boldsymbol{\pi}(R_1), j, s\big) + \mathbb{p}_{R_2} \mathrm{GI}\big(\boldsymbol{\pi}(R_2), j, s\big) \Big]$$

where $\mathbb{p}_{R_1}$ and $\mathbb{p}_{R_2}$ is the fraction of points from the parent node that ends up in, respectively, R_1 and R_2.

Hence, the algorithm proceeds by greedily finding splits that will minimise the Gini index (or the cross-entropy, since both have similar properties). By finding the (j, s) split that minimise such quantities, we obtain regions where the distribution of points across classes is maximally inhomogeneous and hence dominated by data belonging only to one class. You will explore this implementation in your Python notebook.

The splits will continue until a stopping criterion is met. Examples of stopping criteria:

- plateau in the optimisation, i.e., no gain in the quality of the classification;
- small number of points in region, i.e., we establish an *a priori* limit in how finely we will split our regions so that the samples are not too sparse;
- maximum depth of the tree, i.e., we establish *a priori* a limitation on the number of splits;
- and many others derived from practice.

Once the algorithm to estimate the decision tree from the data has terminated, we have our model. To make predictions from the decision tree we have the following:

(1) Given a new data point $\mathbf{x}^{\text{in}}$,
(2) Find $R_{\boldsymbol{\alpha}}$, such that $\mathbf{x}^{\text{in}} \in R_{\boldsymbol{\alpha}}$,
(3) Obtain $\boldsymbol{\pi}(R_{\boldsymbol{\alpha}})$,
(4) $\hat{y} = \hat{f}^{\text{DT}}(\mathbf{x}^{\text{in}}) = \text{argmax}_q \, \boldsymbol{\pi}\big(R_{\boldsymbol{\alpha}}(\mathbf{x}^{\text{in}})\big)$.

In words, find the region in which the new sample lies, and assign the majority class of the samples in that region.

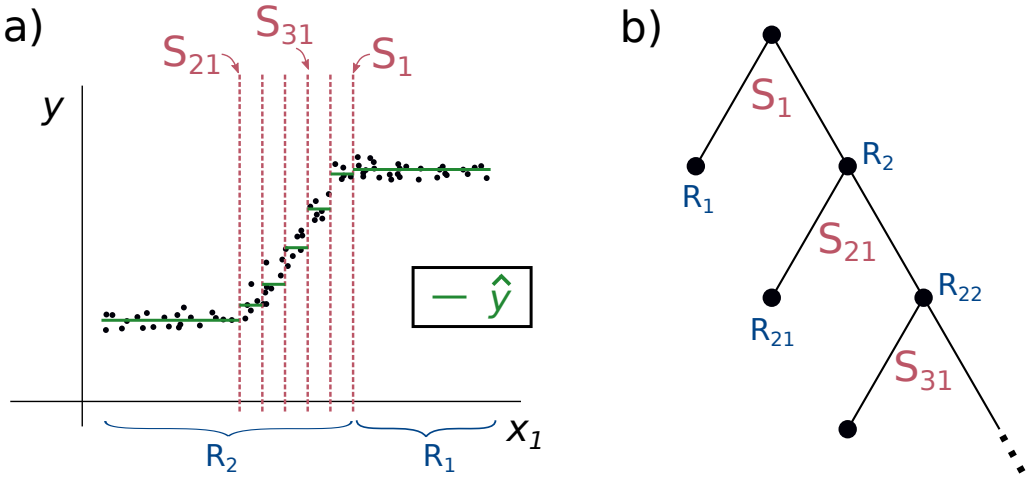


Figure 5.3. Visual intuition of a decision tree in a regression task. **a)** Given some data consisting of one input and one output variable, a decision tree will partition the space of x_1 into chunks where the data is relatively level (this vague expression will be made more precise later on). When given a new input instance, the prediction will correspond to the average of the particular region, shown as green lines. **b)** Similar to the classification case, this can also be drawn as a binary tree where each decision corresponds to one region in the input space.

1.2. Regression task: In this case, our task is to predict the continuous outcome variable $y \in \mathbb{R}$. Much of what we saw in the previous section regarding the intuition and setup of a decision tree still holds for the case of doing regression with decision trees. Just like before, the input space is divided into regions through a series of decisions, which will yield a corresponding binary tree. For some visual intuition, we use the simplest case of one continuous input and one continuous output variable. See Figure 5.3 as well as its caption for a more in-depth explanation. We chose a non-linear data set as an example to highlight the fact that decision trees are good at finding trends and making predictions in highly non-linear data.

The only difference in the procedure lies in the cost function that is optimised at each split. The greedy algorithm will choose (j, s) that minimises the following loss function, called the Residual Sum of Squares (RSS). For a split (j, s) giving rise to regions $R_1(j, s)$ and $R_2(j, s)$ reads:

$$\min_{j,s} RSS(\mathbf{y}; j, s), \qquad RSS(\mathbf{y}; j, s) = \left[\sum_{\mathbf{x}^{(i)} \in R_1(j,s)} (y^{(i)} - \overline{y}_{R_1})^2 + \sum_{\mathbf{x}^{(i)} \in R_2(j,s)} (y^{(i)} - \overline{y}_{R_2})^2 \right]$$

where $\overline{y}_{R_1}$ is the mean of the output variable in region R_1, and similarly for $\overline{y}_{R_2}$:

$$\overline{y}_{R_1} = \frac{\sum_{i=1}^{N} I(x_j^{(i)} < s) \cdot y^{(i)}}{\sum_{i=1}^{N} I(x_j^{(i)} < s)}$$

The splits continue, as indicated above, until a stopping criterion is met (usually plateau in the optimisation, or reaching a pre-established depth of the tree or a number of minimal points in a region).

Finally to obtain a prediction for the regression problem using a decision tree, we compute the average of the outcome variable in the particular region where the new sample lies. In other words, given a new instance $\mathbf{x}^{\text{in}}$, we find the region R_α in which it lies and obtain

$$\hat{y}^{\text{DT}}(\mathbf{x}^{\text{in}}) = \overline{y}_{R_\alpha} \text{ with } \mathbf{x}^{\text{in}} \in R_\alpha$$

2. Random Forests

2.1. Bagging. A major flaw of decision trees is that they tend to overfit. The smallest changes in the input data or any new training data will usually lead to vastly different trees. In order to overcome this shortcoming, we introduce a certain type of randomised algorithms called *ensemble learning/methods*. The idea is that through randomisation, we are able to reduce the variability in the estimator arising from new data. There exist a large variety of ensemble methods, amongst which are those that mix different models or a method called *boosting*.

Here we introduce another specific, possible the most popular type of ensemble learning, namely the *bagging* method, which is short for *Bootstrap aggregating*. The 'bootstrap' is a general procedure for assessing statistical accuracy of quantities estimated from a dataset, based on drawing a set of random samples from the original dataset and repeating the estimation for each of them (see Hastie,Tibshirani, Friedman, *The Elements of statistical learning*, Chap. 7, section 7.11, for more details). Note that here we apply *bagging* to decision trees specifically, but the method can be applied to almost any classification or regression model, and even to different models at the same time.

Starting with our data sample, let's call it $\mathbf{Z} = \{\mathbf{x}^{(i)}, y^{(i)}\}, i = 1, \ldots, N$, we have the following algorithm:

(1) **Bootstrapping:** Produce B random samples $\mathbf{Z}^{*b}$ for $b = 1, \ldots, B$ from $\mathbf{Z}$, all of size $N' \leq N$, by random sampling with replacement. (Sampling with replacement will lead to each $\mathbf{Z}^{*b}$ potentially containing repeated samples and/or not containing certain samples (absent samples) from the original $\mathbf{Z}$).

(2) **Ensemble of models:** In bootstrap methods, one re-obtains the quantity of interest, let's call it $S(\mathbf{Z})$, from each of the $\mathbf{Z}^{*b}$, $b = 1, \ldots, B$ - these are the bootstrap replications $S(\mathbf{Z}^{*b})$. In this case, we re-train a decision tree for each bootstrap sample, getting:

$$\left\{ \hat{f}_b^{\text{DT}} \right\}_{b=1}^{B}$$

In other words, we produce an *ensemble* of models, hence the term *ensemble learning*.

(3) **Aggregating:** We use the ensemble of models to predict our variable:

(a) *Regression:* Given a new sample $\mathbf{x}^{\text{in}}$, we simply obtain the average of all the predictions made by the ensemble of decision trees:

$$\hat{y} = \hat{f}(\mathbf{x}^{\text{in}}) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b^{\text{DT}}(\mathbf{x}^{\text{in}})$$

(b) *Classification:* In this case we will get a collection of B probability vectors, each of them being Q-dimensional:

$$\mathbf{x}^{\text{in}} \to \left[\boldsymbol{\pi}_1^{\text{DT}}, \ldots, \boldsymbol{\pi}_B^{\text{DT}}\right]$$

$$\boldsymbol{\pi}_b = \begin{pmatrix} \pi_{b,1} \\ \vdots \\ \pi_{b,Q} \end{pmatrix}$$

From this collection, we obtain the aggregated, average probability vector:

$$\boldsymbol{\pi} = \frac{1}{B}\sum_{b=1}^{B} \boldsymbol{\pi}_b^{\text{DT}}$$

And finally, our prediction is the class with the highest probability:

$$\hat{y} = \underset{q}{\operatorname{argmax}}\, \boldsymbol{\pi}$$

Whilst this does introduce a certain amount of robustness, in many cases, the bagging method will yield somewhat correlated outcomes, which is usually due to the dominance of some predictor variables. For example, if there is some predictor that is very determining, many decision trees will predominantly split along that variable. This is usually not a good property to have, since some particular predictor may be very good for describing the data at hand, but may not generalise well at all.

Furthermore, if the set of models for a given new instance are correlated, then averaging them will not reduce the variance beyond a limit, even if we increase B. Indeed, if one has B random variables (in this case, the trees), that are simply i.d. (identically distributed, but not necessarily independent), with positive pairwise correlation ρ and each of them with variance σ^2, the variance of their average as an estimator is:

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

(showing this is actually Exercise 15.1 in Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*). Thus, we need to 'uncorrelate' the trees more radically.

2.2. Random Forests (via feature bagging). *Random forests* add an additional level of randomisation, namely using the *random subspace method* (also known as *feature bagging*).

Since much of the correlated outcomes discussed above come from certain predictors being dominant, **we restrict at random the number of predictors among which the split is chosen to $m \leq p$ predictors**. This means that, compared to bagging alone (subsection 2.1), steps (1) and (3) are unchanged, while in step (2) we train every tree only on a subset of m features randomly sampled at each split, producing an ensemble of modified decision trees that are less correlated among them.

Let's summarise schematically the steps for training and evaluating a random forest:

Algorithm: Random Forests for Regression and Classification

Training.
For $b = 1$ to B:

(1) Draw a bootstrap sample $\mathbf{Z}^{*b}$ from the training data.

(2) Train a tree $\hat{f}_b^{\mathrm{DT}}$ on the bootstrapped sample as follows.
For each terminal node at the current level, repeat:
(i): Draw at random a subset of m features from the p features.
(ii): Find the optimal split (j, s) (according to the relevant cost function), choosing j only among the m features randomly sampled in *(i)*.
(iii) Perform the split into two daughter nodes.
Until the stopping criterion is reached.

Making predictions (aggregating). Given a new data point $\mathbf{x}^{\mathrm{in}}$:

(1) *Regression:*

$$\hat{y} = \hat{f}(\mathbf{x}^{\mathrm{in}}) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b^{\mathrm{DT}}(\mathbf{x}^{\mathrm{in}})$$

(2) *Classification:*

$$\hat{y} = \underset{q}{\operatorname{argmax}}\, \boldsymbol{\pi}, \text{ with } \boldsymbol{\pi} = \frac{1}{B}\sum_{b=1}^{B} \boldsymbol{\pi}_b^{\mathrm{DT}}$$

Hyperparameters. There are certain parameters that will need to be decided before running a random forest model. Some of these are the following:

- B = number of trees: There is usually no need to make this excessively big, it tends to converge fairly quickly.
- Minimum leaf size: A stopping criterion that will stop further splits as soon as the leaf reaches a minimum number of instances, with the goal of preventing overfitting. By default, this is usually set to rather small values (1-5).
- m = number of predictors sampled randomly at each split. If this is small, we obtain a large variety of decision trees, guaranteed to not have high correlations and vice versa. The guidelines by the authors of random forests is to choose:

$$\begin{cases} m \approx \frac{p}{3} & \text{for regression} \\ m \approx \sqrt{p} & \text{for classification} \end{cases}$$

Out-of-bag error. Using the bootstrapping method has many advantages, amongst which is the possibility of using *out-of-bag* (OOB) samples for error estimation. For every bootstrap sample $\mathbf{Z}^{*b}$, there are some samples that were not picked: these are OOB samples, and they play the role of unseen samples for the corresponding decision tree, hence we can compute on those an error measure (OOB error) which plays the role of a validation error. This can be useful to perform a cross-validation-type hyper-parametric search without appealing to T-fold cross-validation. To compute the OOB error, the key step is to take, as predictor for the data point $\{\mathbf{x}^{(i)}, y^{(i)}\}$, the average of predictions by only those trees corresponding to bootstrap samples in which $\{\mathbf{x}^{(i)}, y^{(i)}\}$ did not appear.

Interpretability and Variable Importance. A disadvantage of this approach of progressive partitioning of the feature space is that there is a loss of interpretability, in the sense that we lose a direct connection with the predictor variables compared e.g. to linear

regression, where the coefficient values bring information on the relevance of each predictor. But there are ways of gaining interpretability by attributing "importance" to the different predictors towards the prediction of the outcome variable, see Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, Chapters 10.13, 15.3 for more details.

One of these ways uses precisely the OOB samples, as instances of unseen data for each tree in the forest. For each decision tree, we have a set of OOB samples, let's denote them as $\mathbf{X}^{OOB}$ and let's evaluate a metric of performance of the tree on $\mathbf{X}^{OOB}$, calling it M^0. For each feature $j = 1, ..., p$ of which we want to gauge the importance, we perform a randomisation over that feature by permuting the elements of the corresponding column $\mathbf{X}^{OOB}$, in such a way as to destroy its correlation with the other features. We re-compute the metric on this version of $\mathbf{X}^{OOB}$ with the values at column j permuted and we denote its value in this case by $M^0_{j\,\text{permuted}}$. We can take as measure of importance of feature (or variable) j for the tree's prediction the performance gap given by:

$$Importance = |M^0 - M^0_{j\,\text{permuted}}| \qquad \text{for a given tree.}$$

The absolute value ensures that the variable importance is positive (depending on the metric, the randomisation of one feature might lead to higher or lower values of the metric). Variable importance for the random forest is then found by averaging importance over the different trees of the forest; typically, importance is analysed and visualised by expressing it as a percentage of the maximal importance across features (see section 15.3.2 in see Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*).

Final Aside. A powerful alternative to standard random forests is Gradient Boosting Decision Trees (GBDTs), see Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, Chapter 10.10 for more details. Its heuristic is the iterative addition of weak ('easy') models to a strong model aimed at reducing the error of the strong model (i.e., the model we aim to improve through this procedure).

Let's take first the example of a strong model solves a regression task minimising the squared prediction errors (like the MSE). At each iteration t, the strong model is evaluated on the training data and a weak model is trained to predict its *error*. Let's denote the prediction by the weak (respectively, strong) model at iteration t by w_t (respectively S_t). At iteration $t+1$ the strong model is updated by subtracting the weak model's prediction (so as to reduce the strong model's error):

$$S_{t+1} = S_t - \nu w_t \tag{5.3}$$

with a hyperparameter ν called shrinkage. ν is a tunable hyperparameter which plays the role of the learning rate in gradient descent, i.e., it controls the rate of updates of the strong model. The iterations are repeated until a stopping criterion is met (maximum number of iterations, signals of overfitting of the strong model).

More generally, one has the loss function of the supervised learning task under consideration: $L(y, S_t)$, which depends on the output y and the strong model's prediction at iteration t. The weak models are trained iteratively to predict the *gradient* of the loss with respect to the predictions: $w_t = \partial L(y, S_t)/\partial S_t$ and the strong predictions are updated according to (5.3). In the case of a loss given by the squared error $L(y, S_t) = (y - S_t)^2$:

$$\frac{\partial L(y, S_t)}{\partial S_t} = -2(y - S_t)$$

that is, one recovers the signed error as the learning target of the weak model. By requiring the weak model to fit the gradient of the loss that the strong model has been trained to minimise, the update of the strong model follows the direction of steepest descent of the loss.